

A Study on Explainability in RL Models

Joseph (Jack) Bosco
School of Engineering and Applied Science
Columbia University
New York, NY
jab2516

Akshara Pramod
School of Engineering and Applied Science
Columbia University
New York, NY
ap4613

Abstract—This project explores the application of Variational Autoencoders (VAEs) to enhance explainability in reinforcement learning (RL) models applied to the CarRacing-v2 environment. We propose and implement a framework that learns meaningful low-dimensional latent representations from high-dimensional visual inputs, enabling more interpretable and efficient training of RL agents. By training a VAE on frames collected from human gameplay and then using the encoded latent space as the state representation for a Soft Actor-Critic (SAC) agent, we create an architecture that bridges the explainability gap in deep RL systems. Our approach demonstrates how dimensionality reduction through VAEs creates interpretable state representations while maintaining performance. We utilize t-SNE visualization, latent space traversal, policy vector field analysis, and cluster analysis to gain insights into the agent’s decision-making process. Results show that our 2-dimensional latent space effectively captures essential driving features and enables visualization of the agent’s policy, proving the value of this approach for explainability in high-dimensional RL tasks.

Index Terms—VAE, Soft Actor-Critic Agent (SAC), Deep RL

I. INTRODUCTION

Deep reinforcement learning has achieved remarkable success in solving complex tasks, particularly in environments with high-dimensional sensory inputs such as image data. However, most deep RL models operate as black boxes, making it challenging to interpret why certain decisions are made. This lack of transparency poses significant challenges for stakeholders who must trust these automated systems, especially in safety-critical domains like autonomous driving, healthcare, finances, etc.

In this project, we address the explainability challenge in RL by incorporating Variational Autoencoders to learn meaningful latent representations from high-dimensional visual states in the CarRacing-v2 environment. Our approach brings two key advantages: (1) learning a compressed, meaningful state representation that captures essential driving features and (2) enabling interpretable visualization of the agent’s decision-making process.

We aim at demonstrating that a low-dimensional latent space can effectively capture the essential features needed for performing well in a driving task while simultaneously enabling visualization and interpretation of the agent’s policy. This represents a significant step toward explainable RL systems that can be trusted and understood by human operators.

II. RELATED WORK

The intersection of reinforcement learning (RL) and explainability has emerged as a critical research area, particularly for applications in domains where transparency is essential for safety and trust. In this section, we brief about our approach within the existing literature and highlight how our method addresses key limitations of previous work.

Kingma and Welling [1] introduced Variational Autoencoders (VAEs) as a principled approach to learning latent representations of high-dimensional data. Their probabilistic formulation enables both effective compression and generative capabilities. Building on this work, Ha and Schmidhuber [2] developed “World Models,” which use VAEs to learn compact representations of environments for reinforcement learning. While their work focused on using these representations for planning and imagination-based learning, our approach leverages similar VAE-based representations specifically for explainability. Gebauer and Bennewitz [3] explored the use of autoencoders in LiDAR-based navigation but noted potential pitfalls when autoencoders become too powerful. They found that excessive representational capacity can lead to memorization rather than meaningful feature extraction. Our work addresses this concern by constraining the VAE to an extremely low-dimensional latent space (2D), forcing it to learn only the most essential driving features and creating an inherently interpretable representation. Haarnoja et al. [4] developed Soft Actor-Critic (SAC), an off-policy maximum entropy RL algorithm that balances exploration and exploitation through entropy maximization. Unlike traditional RL algorithms that focus solely on reward maximization, SAC incorporates an entropy bonus that encourages exploration of multiple effective strategies. We adopt SAC for our approach due to its sample efficiency and its natural tendency to learn diverse behaviors, which is valuable for exploring the latent space effectively. In the autonomous driving domain, Bukhari et al. [5] propose an explainable AI-based federated deep reinforcement learning approach that incorporates SHAP (SHapley Additive exPlanations) values to explain agent decisions. Their method differs from ours by using local explanations for specific decisions rather than providing global interpretability through state space visualization. While their approach may offer more fine-grained explanations for individual actions, our method provides a more holistic understanding of the agent’s overall

strategy through comprehensive visualization of the entire state-action mapping.

III. METHODS

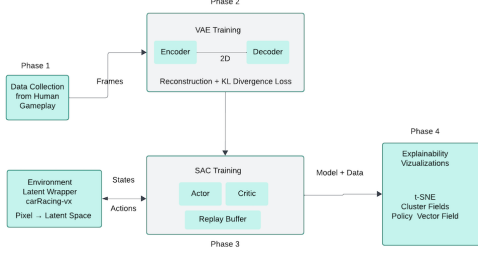


Fig. 1. System Workflow

Our approach integrates three key components to create an explainable reinforcement learning system:

- i. **Variational Autoencoder (VAE) for learning interpretable state representations:** A deep generative model that compresses high-dimensional visual inputs ($96 \times 96 \times 3$ RGB images) into a structured 2-dimensional latent space, capturing essential driving features while enabling direct visualization.
- ii. **Soft Actor-Critic (SAC) algorithm for policy learning:** A maximum entropy reinforcement learning algorithm that learns a driving policy directly in the VAE’s latent space, balancing exploration and exploitation while effectively handling the continuous control task of steering and acceleration.
- iii. **Specialized visualization techniques for explainability:** A combination of complementary methods including policy vector field visualization, t-SNE embedding, cluster analysis, and trajectory tracking that reveal different aspects of the agent’s decision-making process and behavioral patterns.

This section provides the theoretical foundations and conceptual understanding of each component we have used in our implementation.

A. Framework Overview

Deep reinforcement learning systems operating on high-dimensional inputs such as images typically function as black boxes, with internal representations that resist human interpretation. Our framework addresses this explainability challenge by decomposing the learning process into sequential stages. First, we learn a compressed, structured representation of the visual environment using a VAE. Then, we train a reinforcement learning agent directly in this interpretable latent space rather than on raw pixels. Finally, we apply visualization techniques to reveal the agent’s decision-making process. Figure 1 illustrates the overall architecture of our approach.

B. Variational Autoencoders: Theoretical Foundations

Variational Autoencoders [1] represent a class of generative models that learn compact latent representations of high-dimensional data through a principled probabilistic framework. Unlike standard autoencoders that deterministically map inputs to latent codes, VAEs model the underlying probability distribution of the data, enabling more robust representations and generative capabilities.

1) *Probabilistic Formulation:* The VAE framework models the data generation process as involving latent variables z that generate observable variables x through some conditional distribution $p_\theta(x|z)$. The joint distribution $p(x, z)$ is factorized as:

$$p(x, z) = p(z)p_\theta(x|z) \quad (1)$$

Where $p(z)$ is typically a standard normal distribution $\mathcal{N}(0, I)$.

The key challenge in VAEs is inference—computing the posterior distribution $p(z|x)$, which is generally intractable. VAEs address this by introducing an approximate posterior $q_\phi(z|x)$ parameterized by neural networks, and then optimizing the parameters to minimize the difference between the true and approximate posteriors.

2) *Evidence Lower Bound:* VAEs are trained to maximize the Evidence Lower Bound (ELBO):

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p(z)) \quad (2)$$

The first term represents the reconstruction quality—how well the decoder can reconstruct the input from the latent code. The second term is the Kullback-Leibler divergence between the approximate posterior and the prior, serving as a regularization term that encourages the latent space to have a structured distribution.

In our work, the VAE serves as a state abstraction mechanism that compresses the high-dimensional visual input from the CarRacing-v2 environment into a low-dimensional space that captures the essential driving features while enabling direct visualization.

C. Soft Actor-Critic: Theoretical Foundations

Soft Actor-Critic [4] is a state-of-the-art deep reinforcement learning algorithm based on the maximum entropy framework, which augments the standard RL objective with an entropy maximization term. This approach leads to more robust policies that explore more effectively and can better handle uncertainty.

1) *Maximum Entropy Objective:* The standard RL objective seeks to maximize expected cumulative rewards:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \quad (3)$$

SAC extends this with an entropy term to form the maximum entropy objective:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right] \quad (4)$$

Where $\mathcal{H}(\pi(\cdot|s_t))$ represents the entropy of the policy at state s_t , and α is a temperature parameter controlling the trade-off between reward maximization and entropy maximization.

2) *Actor-Critic Architecture*: SAC employs an actor-critic architecture with several key components:

- i. A policy network (actor) $\pi_{\phi}(a|s)$ that outputs a distribution over actions
- ii. Twin Q-networks (critics) $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$ that estimate expected returns
- iii. A temperature parameter α that balances exploration and exploitation

The critics are trained to minimize the Bellman error, while the actor is trained to maximize the expected Q-value while maintaining high entropy:

$$J_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} [\mathbb{E}_{a \sim \pi_{\phi}(\cdot|s)} [Q_{\theta}(s, a) - \alpha \log \pi_{\phi}(a|s)]] \quad (5)$$

In our framework, SAC learns directly in the VAE's latent space, taking advantage of the compact, structured representation to learn effective driving policies more efficiently while maintaining interpretability.

D. Explainability Methods

Explainable AI (XAI) methods seek to make AI systems' decision-making processes transparent and interpretable to humans. In reinforcement learning, explainability is particularly challenging due to the sequential nature of decision-making and the complexity of the learned policies.

1) *Dimensionality Reduction and Clustering*: Two foundational techniques for explainability are dimensionality reduction and clustering, the approaches that we have used in our work:

- i. **Dimensionality reduction**: Methods like t-SNE (t-distributed Stochastic Neighbor Embedding) [6] project high-dimensional data into lower-dimensional spaces while preserving local structure. t-SNE minimizes the divergence between probability distributions that represent similarities in the original and reduced spaces, making it particularly effective at revealing clusters.
- ii. **Clustering**: Techniques like K-means [7] partition the state space into distinct groups based on similarity, revealing behavioral modes in the agent's strategy. K-means minimizes the sum of squared distances between points and their assigned cluster centers.

2) *Vector Fields and Trajectory Analysis*: For understanding policies and behaviors over time, we have used vector fields and trajectory analysis:

- i. **Vector fields**: Visualizations that show the agent's action at every point in the state space, revealing the flow of the policy. These are analogous to force fields in physics, showing how the agent would act from any possible state.

- ii. **Trajectory analysis**: The study of state-action sequences over time, revealing how the agent navigates through the state space. This approach draws from dynamical systems theory, where phase portraits show how systems evolve.

In our framework, these explainability methods transform the traditionally opaque deep RL system into a transparent one, where the agent's perceptions, decisions, and strategies can be directly observed and analyzed.

IV. EXPERIMENTS

A. Dataset Collection and Preprocessing

We collected a comprehensive dataset specifically for training our VAE model. This dataset consisted of 2,000 RGB frames with dimensions $96 \times 96 \times 3$ captured from human gameplay sessions in the CarRacing-v2 environment [8]. Human gameplay was selected over random exploration to ensure the dataset contained meaningful driving behaviors that would be encountered during actual policy execution.

1) *Data Acquisition Process*: The data collection process involved:

- i. Recording approximately 10 complete human gameplay episodes using the Gymnasium render functionality
- ii. Sampling frames at regular intervals (every 3 timesteps) to ensure diversity while maintaining temporal coherence
- iii. Storing raw frames as PNG files to preserve image quality
- iv. Ensuring coverage of various track configurations, turn types, and driving scenarios

2) *Data Augmentation*: To enhance the diversity and robustness of our training dataset, we implemented the following augmentation techniques:

- i. **Region-specific cropping**: We identified that pixels 0-83 contained the static toolbar in the CarRacing-v2 environment. This region was cropped to focus the model on the dynamic driving area.
- ii. **Horizontal flipping**: We applied horizontal mirroring only to the relevant gameplay region above the toolbar. This effectively doubled our dataset size while creating balanced representations of left and right turning scenarios, which is critical for learning symmetrical driving behaviors.

These augmentation techniques expanded our effective dataset size to approximately 4,000 frames, providing sufficient variety for the VAE to learn a robust latent representation.

3) *Preprocessing Pipeline*: Our preprocessing pipeline consisted of the following steps:

- i. Conversion from RGB to normalized floating-point tensors in range [0,1]
- ii. Application of augmentation techniques
- iii. Construction of a PyTorch Dataset class with custom transformations [9]
- iv. Creation of data loaders with batch size 64 for efficient training

The dataset was partitioned using `torch.utils.data.random_split` with fixed random seeds (42) to ensure reproducibility:

- i. 80% training set (approximately 3,200 augmented frames)
- ii. 10% validation set (approximately 400 augmented frames)
- iii. 10% test set (approximately 400 augmented frames)

B. VAE Training

Our VAE implementation used a convolutional architecture specifically designed for the CarRacing-v2 environment, as demonstrated in the Figure 2. below.

Encoder	Decoder
Input: 320 × 320 local image	Input: latent sample $\in \mathbb{R}^k$
Conv. 32 × 3 × 3, stride 2 ReLU, BN	Dense, 256, ReLU
Max pool. 2 × 2	Dense, 5 × 5 × 128, ReLU reshaped
Conv. 32 × 3 × 3, stride 2 ReLU, BN	Trans. Conv., 128 × 3 × 3 stride 2, ReLU
Conv. 64 × 3 × 3, stride 1 ReLU, BN	Trans. Conv., 64 × 3 × 3 stride 2, ReLU
Avg. pool. 2 × 2	Trans. Conv., 64 × 3 × 3 stride 2, ReLU
Conv. 128 × 3 × 3, stride 2 ReLU, BN	Trans. Conv., 32 × 3 × 3 stride 2, ReLU
Conv. 128 × 3 × 3, stride 2 ReLU, BN, flatten	Trans. Conv., 32 × 3 × 3 stride 2, ReLU
Dense, 256, ReLU	Trans. Conv., 32 × 3 × 3 stride 2, ReLU
Dense, 2k	Conv. 1 × 3 × 3, stride 1
Output: Diag. Gaussian	Output: Ind. Bernoulli

Fig. 2. VAE Architecture

1) *Implementation Details:* We implemented the VAE in PyTorch 1.x and optimized it using Adam optimizer with learning rate 1×10^{-3} and momentum parameters $\beta_1 = 0.9$, $\beta_2 = 0.999$. The model was trained for 200 epochs with batch size 64. To ensure reproducibility and track progress, model weights were checkpointed every 20 epochs. We displayed early validation reconstructions periodically during training to monitor for potential overfitting and ensure the model was learning meaningful representations.

The BatchNorm2d layers after convolutional operations were critical for training stability, especially given the relatively small dataset size. The pooling operations helped reduce spatial dimensions efficiently while maintaining feature representation quality.

2) *Rationale for Pretraining:* By learning an unsupervised world model of the CarRacing-v2 frames, we were able to extract features that capture essential track geometry and car orientation. These compact latent features greatly reduce the state dimensionality for the downstream RL agent, from the original 27,648 dimensions (96×96×3) to just 2 dimensions.

This approach offered two key advantages: First, it significantly improved sample efficiency for reinforcement learning by providing a structured, low-dimensional state space. Second, it enabled direct interpretability of the agent’s behavior by creating a visualizable representation of the environment state, such latent representations can be highly effective for understanding how generative models conceptualize their environment.

C. Environment Wrapper

To integrate the trained VAE with the reinforcement learning agent, we developed a custom wrapper for the CarRacing-v2 environment [8]:

- **LatentPassthroughExtractor:** A feature extractor class compatible with Stable Baselines3 that passes the latent vectors directly to the policy network without further processing.
- **CarRacingLatentWrapper:** A Gymnasium wrapper that encodes RGB observations through the VAE encoder before passing them to the agent and implements reward shaping.

D. SAC Training

We implemented the Soft Actor-Critic algorithm using the Stable Baselines3 library with several customizations:

1) Environment Setup:

- Base environment: CarRacing-v2 with no rendering during training
- Wrapper: CarRacingLatentWrapper with our pre-trained VAE
- RecordEpisodeStatistics wrapper for performance tracking
- DummyVecEnv for compatibility with Stable Baselines3
- VecNormalize for observation and reward normalization

2) Policy Configuration:

- Policy type: MlpPolicy
- Feature extractor: LatentPassthroughExtractor
- Network architecture: 2 hidden layers with 256 units each
- Activation function: ReLU
- Output distribution: Gaussian with state-dependent mean and standard deviation

3) *Training Hyperparameters:* We initialized the SAC agent with carefully tuned parameters for our specific task:

```
model = SAC(
    "MlpPolicy",
    venv,
    policy_kwargs=policy_kwargs,
    learning_rate=3e-4,
    buffer_size=200_000,
    batch_size=256,
    tau=0.005,
    gamma=0.99,
    train_freq=1,
    gradient_steps=1,
    verbose=1,
    device="cuda" if torch.cuda.is_available()
    else "cpu" #NVIDIA T4 GPU used
```

4) Evaluation and Checkpointing:

- Evaluation frequency: Every 20,000 timesteps
- Number of evaluation episodes: 5
- Evaluation deterministic: True (using mean action)
- Checkpoint saving frequency: Every 100,000 timesteps
- Best model saving: Based on mean evaluation reward

The SAC agent was trained for a total of 1,000,000 timesteps, which took approximately 8 hours on our hardware setup. We monitored training progress using TensorBoard logs, tracking episode rewards, episode lengths, and policy entropy.

E. Explainability Analysis

1) *Data Collection*: For our explainability analysis, we collected 5,000 state-action pairs from the trained agent. We created a testing environment with the pre-trained VAE and loaded the best SAC model identified during training. For each interaction, we recorded the 2D latent state vector, agent’s actions (steering and acceleration), critic’s Q-value estimates, rewards received, and termination flags. This comprehensive dataset provided the foundation for our visualization techniques.

2) *T-SNE Visualization*: We applied t-SNE dimensionality reduction (perplexity=30, 3,000 iterations) to visualize relationships between states and actions. Four visualizations were created using the same embedding but with different coloring schemes: steering actions (coolwarm colormap), acceleration values (viridis colormap), estimated Q-values (plasma colormap), and rewards (RdYlGn colormap). These visualizations revealed how different state regions correspond to specific driving behaviors and value estimates.

3) *Cluster Analysis*: K-means clustering (k=5) was applied to identify distinct behavioral modes in the latent space. For each cluster, we calculated mean steering and acceleration values to determine the characteristic behavior represented. The analysis revealed specialized clusters for different driving maneuvers: left turns, right turns, straightaways, and transitional states. This clustering provided an interpretable taxonomy of the agent’s behavioral repertoire.

4) *Policy Vector Field*: We created a comprehensive policy visualization by sampling the agent’s actions across a uniform 20x20 grid spanning the latent space (-3 to +3 in both dimensions). At each point, we queried both the policy and critic networks to obtain actions and value estimates. The resulting vector field visualization used steering values to determine flow direction and Q-values for coloring, with matplotlib’s streamplot function rendering the field. This visualization revealed the global structure of the agent’s policy across the entire state space.

5) *Trajectory Visualization*: We recorded three complete episodes in the latent space to understand the agent’s temporal behavior. Each episode trajectory was plotted with a distinct color, with special markers for starting and ending points. Background points representing previously explored states provided context for the trajectories. This visualization revealed how the agent chains behaviors over time, showing cyclical patterns corresponding to lap completion and transitions between different driving modes.

V. RESULTS

A. Agent Performance

Our SAC agent, trained on the 2-dimensional latent representation, achieved impressive performance in the CarRacing-v2 environment. During final evaluation, the agent demonstrated:

This performance confirms that our 2D latent representation captured sufficient information for effective driving. The agent

TABLE I
AGENT PERFORMANCE METRICS

Metric	Value
Average Reward	997.43 $\pm$ 64.79
Min/Max Reward	910.61/1066.22
Average Episode Length	1000.00 steps (maximum)
Average Tiles Visited	224.67

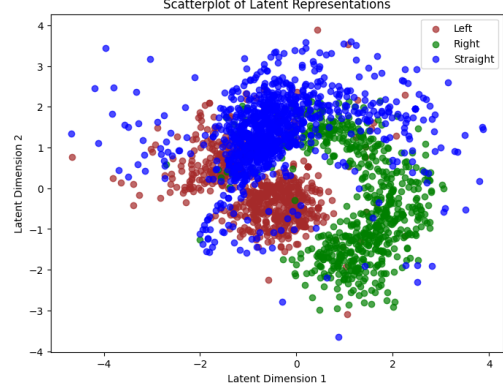


Fig. 3. Scatter Plot of Latent Representations in 2D VAE Space

consistently completed tracks with high rewards, approaching the theoretical maximum score of around 1000.

B. VAE Latent Space Organization

The VAE achieved strong reconstruction performance and learned a highly structured latent space that organizes driving behaviors in a meaningful way. Fig 3. visualizes the 2D latent representations of driving frames, colored by the corresponding driving action. The plot reveals distinct clusters for left turns (red), right turns (green), and straight driving (blue), with minimal overlap between these behavioral modes.

This natural separation in the latent space demonstrates that our 2D bottleneck successfully captured the underlying structure of the driving task without explicit supervision on driving behaviors. The organization of the latent space directly facilitates policy learning, as the agent can learn different actions for different regions of this well-structured representation.

C. Latent Policy Analysis

Figure 4. displays the learned policy structure across the latent space through three complementary visualizations. The steering policy (left) shows a clear separation between left-turning regions (blue) and right-turning regions (red), with

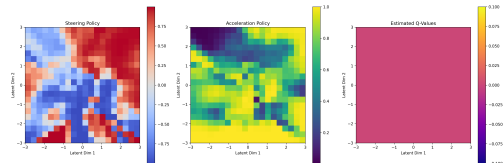


Fig. 4. Latent Policy Analysis Grid

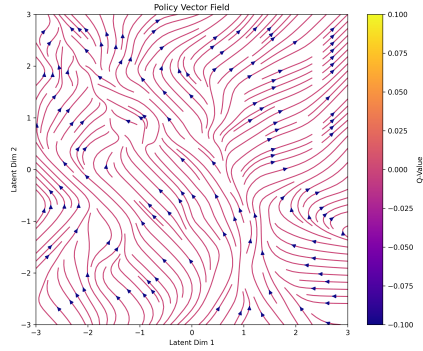


Fig. 5. Policy Vector Field

smooth transitions between them. The acceleration policy (center) reveals areas of high acceleration (yellow) typically corresponding to straightaways and moderate acceleration (green/blue) in turning regions. The Q-value distribution (right) shows relatively uniform values across the state space, indicating the agent has learned to maintain consistent performance in different driving scenarios.

Figure 5. presents the policy vector field, visualizing action flow across the latent space. The streamlines reveal organized patterns corresponding to different driving behaviors:

- Parallel flow lines in the right portions representing consistent forward driving
- Curved flows indicating turning maneuvers
- Converging patterns showing recovery behaviors when approaching track edges

The coherent structure of this vector field demonstrates that the agent has learned a smooth, continuous policy mapping different regions of the latent space to appropriate driving actions.

D. Training Dynamics

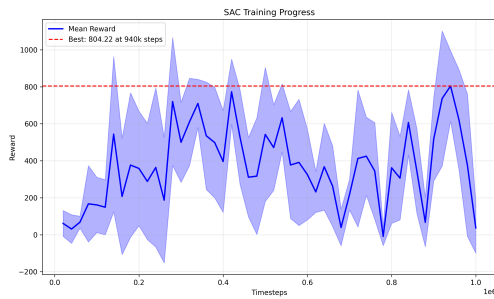


Fig. 6. Training

Figure 6. illustrates the agent's training progression over 1 million timesteps. The blue line shows mean evaluation reward, with the shaded area representing standard deviation. Training exhibited several phases:

- Initial rapid improvement (0-200k steps)

- Alternating plateaus and breakthroughs (200k-600k steps)
- Variable performance with an upward trend (600k-940k steps)
- Peak performance of 804.22 at 940k steps (red dashed line)

The high variance throughout training reflects the procedurally generated nature of the CarRacing-v2 environment, with some tracks being inherently more challenging than others.

E. Behavioral Analysis

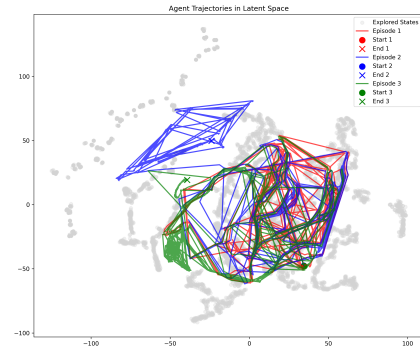


Fig. 7. Trajectory Visualization

Figure 7. visualizes agent trajectories through the latent space during three complete episodes. The colored lines (red, blue, green) show the agent's path through state space, with gray dots representing previously explored states. These trajectories reveal:

- Cyclic patterns corresponding to lap completion
- Shared regions across episodes indicating consistent driving strategies
- Occasional deviations representing adaptive responses to track variations

The trajectories demonstrate that despite being trained in a mere 2D latent space, the agent develops coherent, structured driving behavior that adapts to different track configurations while maintaining a consistent overall strategy.

F. Behavioral Clustering

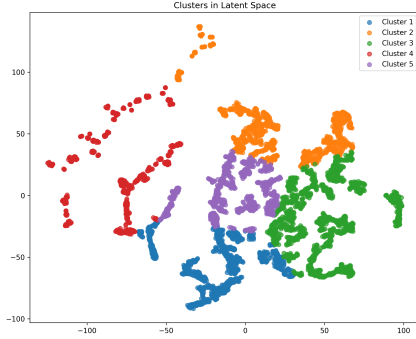


Fig. 8. Clustering Analysis

Figure 8. presents the results of K-means clustering ($k=5$) on states visited by the agent, revealing distinct behavioral modes:

- Cluster 1 (Blue): Left turning maneuvers (average steering: -0.543) with high acceleration (0.792)
- Cluster 2 (Orange): Right turning maneuvers (average steering: 0.533) with moderate acceleration (0.548)
- Cluster 3 (Green): Nearly straight driving (average steering: -0.141) with lower acceleration (0.463)
- Cluster 4 (Red): Moderate left turns (average steering: -0.428) with high acceleration (0.854) and significant braking (0.828), likely representing controlled cornering
- Cluster 5 (Purple): Mild course corrections (average steering: -0.115) with steady acceleration (0.738)

This clustering provides an interpretable taxonomy of the agent’s behavioral repertoire, demonstrating how it specializes actions for different driving scenarios.

G. State-Action Relationships

Figure 9. visualizes the t-SNE embedding of states colored by different attributes. The top-left plot (steering) shows clear separation between left-steering (blue) and right-steering (red) regions. The top-right plot (acceleration) reveals varying acceleration intensity across different state regions, with highest values (yellow) in straightaway sections. The bottom-left plot (Q-value) shows relatively uniform value estimates across the state space, while the bottom-right plot (reward) displays predominantly positive rewards with subtle variations in magnitude.

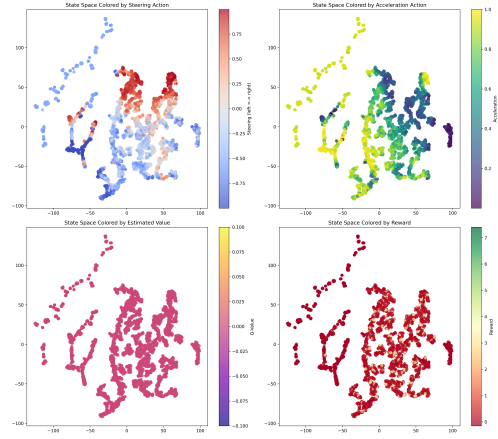


Fig. 9. t-SNE Embeddings

These visualizations reveal relationships between states and actions, confirming that the agent has learned appropriate action mappings across the state space. The clear structure in these relationships demonstrates that despite the extreme dimensionality reduction, the agent has developed a coherent and interpretable driving policy.

VI. CONCLUSION

Our work demonstrates that effective explainability in reinforcement learning can be achieved through dimensionality reduction and visualization of the agent’s latent state space. By compressing high-dimensional visual inputs to just two dimensions using a VAE, we created an interpretable representation that enabled both effective policy learning and comprehensive visualization of the agent’s decision-making process.

The experimental results show that our approach successfully balances performance and explainability. The agent achieved impressive rewards (average 997.43 ± 64.79) while maintaining interpretability through various visualization techniques. The policy vector fields, trajectory analysis, and behavioral clustering provide complementary perspectives on the agent’s behavior, revealing how it navigates the driving task through specialized behavioral modes.

Our work challenges the common assumption that deep reinforcement learning systems must function as black boxes. By designing the architecture with explainability as a primary consideration rather than as an afterthought, we created a system that is both high-performing and transparent. This approach represents a significant step toward more trustworthy AI systems that can be deployed in safety-critical domains.

VII. TEAM DISTRIBUTION

Team Member	Contributions
Jack Bosco	Dataset Collection and Preparation, VAE Training and Latent Representation, Co-write The Report
Akshara Pramod	RL Training : SAC Agent, Explainability Visualizations, Co-write the Report

REFERENCES

- [1] Kingma, D. P., Welling, M. (2013). "Auto-Encoding Variational Bayes." arXiv preprint arXiv:1312.6114
- [2] Ha, D., Schmidhuber, J. (2018). "World Models." arXiv preprint arXiv:1803.10122
- [3] Gebauer, C., Bennis, M. (2021). "The Pitfall of More Powerful Autoencoders in Lidar-Based Navigation." arXiv preprint arXiv:2102.02127
- [4] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S. (2018). "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." ICML 2018
- [5] Bukhari, S. H. R., Dharmasena, S., Mitra, K., Ali, L., & Kumar, N. (2022). "Explainable AI-based Federated Deep Reinforcement Learning for Trusted Autonomous Driving." IEEE Internet of Things Journal
- [6] Maaten, L. V. D., & Hinton, G. (2008). "Visualizing data using t-SNE." Journal of Machine Learning Research, 9(Nov), 2579-2605
- [7] Scikit-learn Documentation: K-means Clustering. <https://scikit-learn.org/stable/modules/clustering.html#k-means>
- [8] Oleg Klimov. Gymnasium Documentation. en. URL: https://gymnasium.farama.org/environments/box2d/car_racing.html
- [9] torch.utils.data — PyTorch 2.7 documentation. URL: <https://pytorch.org/docs/stable/data.html>
- [10] <https://stable-baselines3.readthedocs.io/en/master/modules/sac.html>
- [11] Tom White. Sampling Generative Networks. arXiv:1609.04468 [cs]. Dec. 2016. DOI: 10.48550/arXiv.1609.04468. URL: <http://arxiv.org/abs/1609.04468>
- [12] Puiutta, E., & Veith, E. M. (n.d.). Explainable Reinforcement Learning: A Survey. arXiv.org. <https://arxiv.org/abs/2005.06247>
- [13] Ioffe, S., & Szegedy, C. (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." arXiv preprint arXiv:1502.03167.
- [14] Gymnasium wrapper tutorial: https://gymnasium.farama.org/tutorials/gymnasium_basics/implementing_custom_wrappers/